

Part A §2 — Worker Crash Recovery

Scenario: `edge-worker-0` crashes mid-inference due to a Windows access violation (`0xC0000005`). A client request is in-flight.

Architecture Context

This document covers the recovery path for a single inference worker within the Edge Agent's supervisor-tree process model (defined in Part A §1). The relevant components:

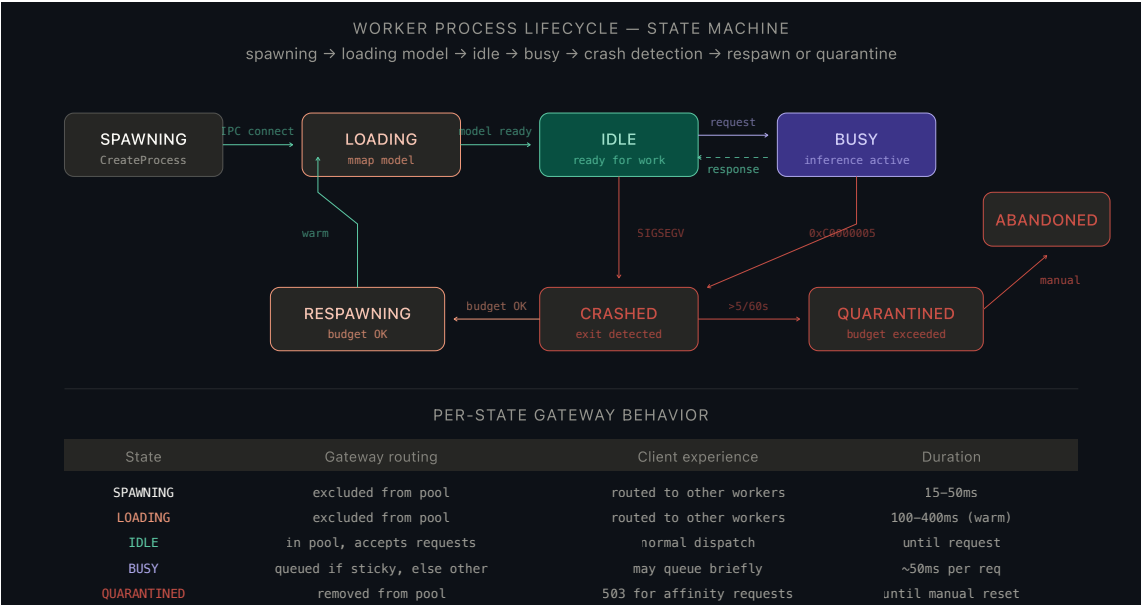
- **edge-supervisor** — persistent parent process that monitors all child PIDs via `WaitForSingleObject` (Windows) / `waitpid` (macOS). Responsible for crash detection, restart budget enforcement, and respawn.
- **edge-gateway** — stateless API server on `127.0.0.1:8741` . Routes requests to workers via sticky model-affinity. Detects broken pipes when a worker crashes.
- **edge-worker-N** — inference worker process. Loads one model at a time. Communicates with the gateway via Named Pipes (Windows) / Unix Domain Sockets (macOS) using MessagePack encoding.

Assumptions

1. **OS page cache retains model data after process exit.** The warm restart path depends on `mmap` - loaded model files staying in the page cache. Under normal memory pressure on laptop-class hardware (16-32GB RAM), this holds for files accessed within the last few seconds.
2. **Client retries are idempotent by `request_id` .** The gateway deduplicates retried requests using the `request_id` field. A client retrying after a 503 will not cause duplicate inference.
3. **Restart budget state persists to disk** via `~/.edge/state/restart_budget.json` . This survives supervisor restarts, preventing a crash-loop scenario where the supervisor itself restarts and loses crash history.
4. **Model files are not modified between crash and restart.** The warm restart skips SHA-256 verification, relying on the fact that model files in `~/.edge/models/` are immutable once deployed.
5. **Inference is stateless.** No partial results, conversation state, or checkpoints need to be recovered. A crashed request is lost, and the client retries from scratch.

Worker Process Lifecycle

The worker process follows a 7-state lifecycle. The supervisor and gateway use this state to make routing and restart decisions.



States

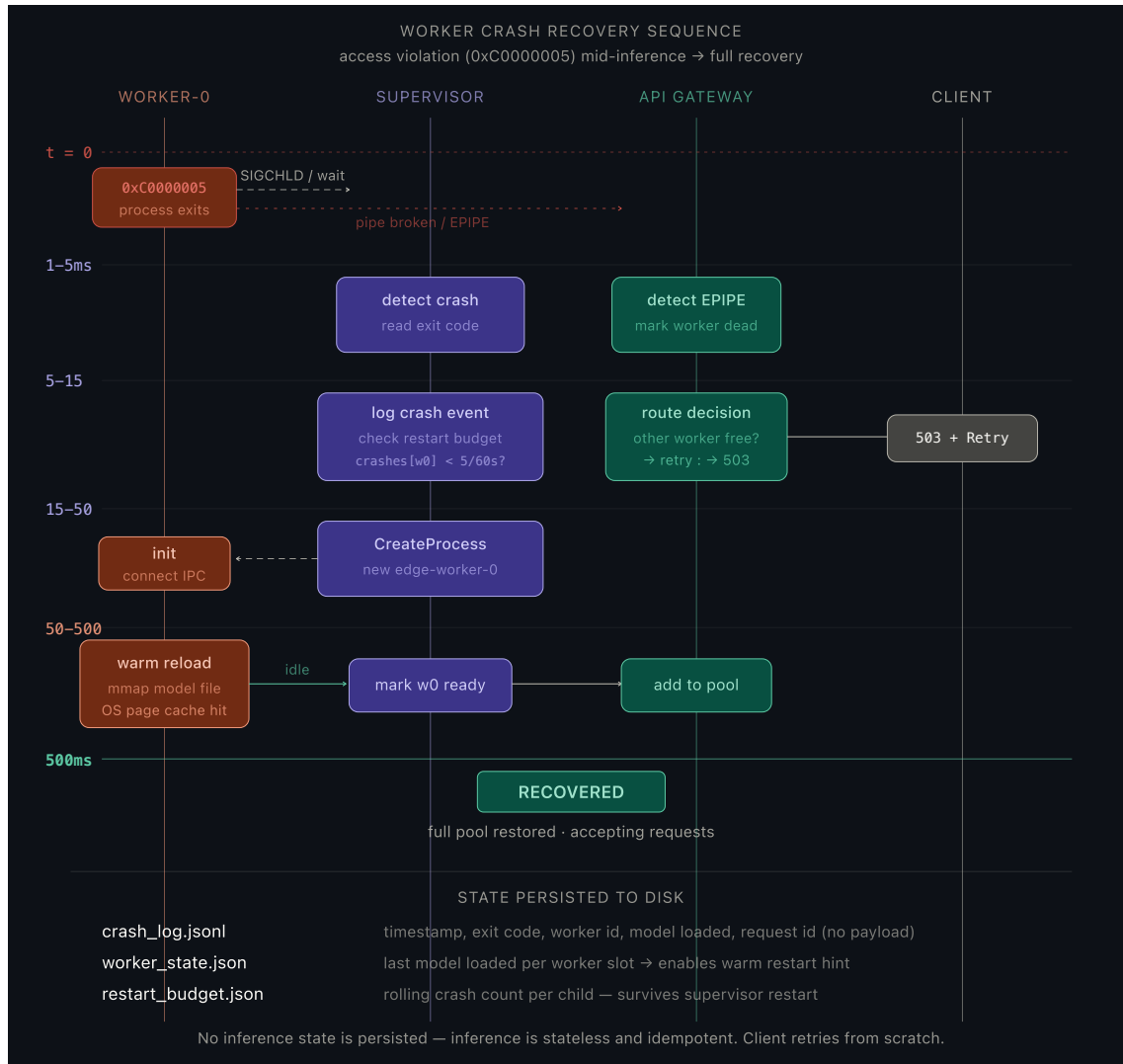
State	Description	Gateway routing	Duration
SPAWNING	CreateProcess / posix_spawn called, waiting for IPC connect	Excluded from pool	15-50ms
LOADING	Connected via IPC, loading model via mmap	Excluded from pool	100-400ms (warm), 2-5s (cold)
IDLE	Model loaded, heartbeat idle, ready for inference	In pool, accepts requests	Until request arrives
BUSY	Inference in progress	In pool, queued if sticky	~50ms per request
CRASHED	Exit detected by supervisor (waitpid / WaitForSingleObject)	Removed from pool, in-flight request fails	~1-5ms detection
RESPAWNING	Budget check passed, new process being created	Excluded from pool	15-50ms
QUARANTINED	Budget exceeded (>5 crashes / 60s)	Removed from pool, 503 for affinity requests	Until manual reset

Transitions

SPAWNING —IPC connect—> LOADING
LOADING —model ready (heartbeat: idle)—> IDLE
IDLE —request dispatched—> BUSY
BUSY —response sent—> IDLE

IDLE or BUSY —SIGSEGV / 0xC0000005—> CRASHED
 CRASHED —budget OK—> RESPAWNING
 CRASHED —budget exceeded—> QUARANTINED
 RESPAWNING —CreateProcess—> SPAWNING (new PID)

Recovery Sequence (0 → 500ms)



t = 0ms — The Crash

The inference runtime (ONNX Runtime / llama.cpp) dereferences an invalid pointer inside the worker process. Windows raises `STATUS_ACCESS_VIOLATION` (`0xC0000005`). The process terminates immediately — there is no opportunity for a graceful shutdown.

Two things happen simultaneously:

1. **The supervisor detects the exit.** It holds a process handle via `WaitForSingleObject` (Windows) or `waitpid` (macOS). The wait unblocks and returns the exit code. This is kernel-mediated — latency is ~1ms.

2. **The API gateway detects a broken pipe.** The IPC channel (Named Pipe / Unix Domain Socket) between the gateway and `worker-0` breaks. The gateway's next `WriteFile` / `write` call returns `ERROR_BROKEN_PIPE` / `EPIPE`. This is detected within the same event loop tick — ~1-3ms.

t = 1-5ms — Detection

Supervisor reads the exit code from the OS:

```
GetExitCodeProcess(worker0_handle, &exit_code)
// exit_code = 0xC0000005 (ACCESS_VIOLATION)
```

It classifies this as an abnormal exit (non-zero) and begins the restart decision.

Gateway marks `worker-0` as dead in its routing table. The in-flight request that was dispatched to this worker is now unresolvable — the response will never arrive.

t = 5-15ms — Decision Phase

Supervisor performs three checks in order:

Check	Question	Action if failed
Restart budget	Has <code>worker-0</code> crashed < 5 times in the last 60s?	If exceeded: don't restart, emit health alert, log to <code>crash_log.jsonl</code>
Exit code	Is this a known transient crash (access violation, stack overflow)?	If unknown/repeated: still restart, but increment a "suspicious" counter
Resource availability	Is there enough free memory to spawn a new worker?	If not: defer restart, notify gateway to reduce queue depth

Assuming the budget check passes (this is the first crash), the supervisor proceeds to respawn.

Gateway makes a routing decision for the in-flight request:

- **If another worker is idle** (e.g., `worker-1` exists and has the same model loaded): re-dispatch the request there. The client sees extra latency (~50ms) but gets a successful response. No error returned.
- **If no other worker is available:** return `503 Service Unavailable` to the client with a `Retry-After: 1` header. The client is expected to retry.

The gateway does **not** buffer the original request payload to disk. The request is small (text prompt + parameters, typically < 1KB) and the client already has it — retrying is cheaper than persisting.

t = 15-50ms — Respawn

The supervisor calls `CreateProcess` (Windows) / `posix_spawn` (macOS) to start a new `edge-worker-0`. The new process:

1. Initializes the inference runtime (ONNX Runtime session, CUDA context if GPU)
2. Connects to the pre-existing IPC pipe/socket — the supervisor reuses the same channel name (`\\.\pipe\edge-worker-0`), so no new pipe creation is needed

3. Reads the **warm restart hint** from the supervisor: "load model `indic-tts-v3` " (the model the old worker had loaded)

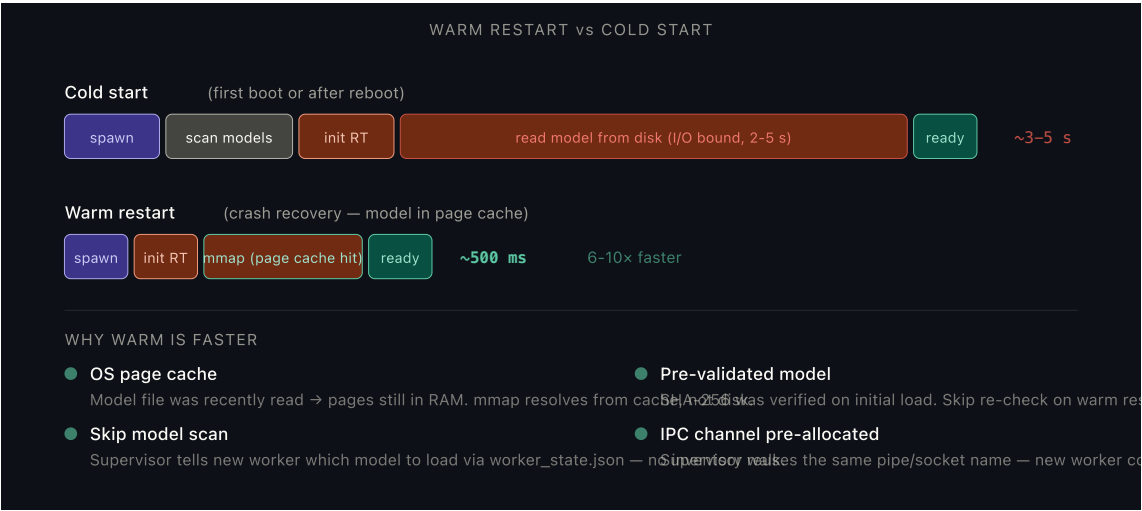
t = 50-500ms — Warm Model Reload

This is where the warm restart advantage kicks in. The new worker loads the model file via `mmap` (macOS) / `CreateFileMapping` + `MapViewOfFile` (Windows).

The model file is already in the OS page cache. The previous worker had it mapped into memory moments ago. The OS doesn't evict page cache contents on process exit — only under memory pressure. So `mmap` resolves the entire file from RAM, not disk.

Operation	Cold start	Warm restart
Model file I/O	2-5s (disk read)	~100ms (page cache hit)
SHA-256 verify	~200ms	Skipped (verified on first load)
Model inventory scan	~20ms	Skipped (hint from supervisor)
Runtime init	~50ms	~50ms (same)
IPC connect	~5ms	~2ms (pipe already exists)
Total	2-5s	~200-500ms

At `t ≈ 500ms` , the new worker sends its first heartbeat with state `idle` . The supervisor marks it ready. The gateway adds it back to the routing pool.



What the API Server Responds to the In-Flight Client

The response depends on pool depth:

Pool state	Gateway response	Client experience
------------	------------------	-------------------

Other worker available with same model	Re-dispatch transparently	~50ms extra latency, no error
Other worker available, different model	Re-dispatch (triggers model swap on that worker)	~2-5s extra latency, no error
No other workers	503 Service Unavailable + Retry-After: 1	Client retries after 1s. By then, warm restart is complete.

The gateway also sets a response header on the next successful request after recovery:

```
X-Edge-Worker-Recovered: true
```

This lets enterprise clients that care about reliability track recovery events without parsing logs.

State Persisted to Disk

Inference is stateless — no partial results, no conversation state, no checkpoints. But three files are updated:

1. `~/.edge/logs/crash_log.jsonl`

Append-only, one JSON line per crash:

```
{ "ts": "2025-01-15T10:23:45.123Z", "worker": "worker-0", "exit_code": "0xC0000005", "model": "indic-tts-v3", "request_id": "req_7f3a", "uptime_ms": 847200 }
```

Used for: post-mortem diagnostics, telemetry (if opted in), detecting crash patterns.

Not included: request payload, user data, or inference results. These are not useful for debugging native crashes and could contain sensitive enterprise data.

2. `~/.edge/state/worker_state.json`

Updated by the supervisor every time a worker loads a new model:

```
{  "worker-0": { "model": "indic-tts-v3", "loaded_at": "2025-01-15T10:00:00Z" },  "worker-1": { "model": "indic-nmt-v2", "loaded_at": "2025-01-15T09:45:00Z" } }
```

Used for: warm restart hints. When the supervisor respawns `worker-0`, it reads this file and tells the new worker to load `indic-tts-v3` directly, skipping the model discovery step.

3. `~/.edge/state/restart_budget.json`

Rolling crash counter per child:

```
{  "worker-0": { "crashes": [1736934225123, 1736934225123], "budget": 5, "window_ms":
```

```
60000 },  
  "worker-1": { "crashes": [], "budget": 5, "window_ms": 60000 }  
}
```

Used for: restart budget enforcement. Persisted so that if the **supervisor itself** restarts (e.g., after an update), it doesn't lose the crash history and accidentally restart-loop a chronically crashing worker.

Why Warm Restart is Faster Than Cold Start

Factor	Cold start	Warm restart	Why
Model I/O	Disk read (2-5s)	Page cache hit (~100ms)	OS keeps recently-read pages in RAM. mmap resolves from cache.
Model verification	SHA-256 check (~200ms)	Skipped	Model was already verified before the crash. File hasn't changed.
Model discovery	Scan ~/.edge/models/ (~20ms)	Skipped	Supervisor passes model ID from worker_state.json.
IPC setup	Create new pipe (~5ms)	Connect to existing pipe (~2ms)	Supervisor pre-creates pipe with the same name.
Runtime init	Same (~50ms)	Same (~50ms)	No shortcut — must re-init ONNX/CUDA from scratch.

The dominant factor is **model I/O**. Cold start reads 500MB-2GB from SSD. Warm restart reads the same bytes from RAM (page cache) at ~10GB/s instead of ~500MB/s. That's the 6-10x speedup.

Scheduler Behavior During Recovery

When a worker crashes, the gateway's scheduler must adapt immediately:

1. **Remove dead worker from routing table** — the sticky affinity map entry for `worker-0` is deleted. Requests for its model are re-routed.
2. **Drain in-flight queue entries** — any queued requests targeting `worker-0` are re-assigned to the next available worker (or returned as 503 if none).
3. **Temporary capacity reduction** — with one fewer worker, the effective throughput drops. The scheduler reduces the backpressure threshold proportionally (e.g., 64 → 32 if 1 of 2 workers is down).
4. **Re-register on recovery** — when the new worker sends its first `idle` heartbeat, the scheduler adds it back and restores the normal threshold.

API Contract: Client Experience During Crash

The client sees one of three outcomes for the in-flight request:

Case 1: Transparent retry (another worker available)

```
HTTP/1.1 200 OK
X-Edge-Compute: ane
X-Edge-Worker-Recovered: true
X-Edge-Latency-Factor: 1.0

{"result": "...", "_meta": {"compute": "ane", "retry_count": 1}}
```

Client sees ~50ms extra latency. No error. The `X-Edge-Worker-Recovered: true` header is informational.

Case 2: 503 with Retry-After (no workers)

```
HTTP/1.1 503 Service Unavailable
Retry-After: 1

{"error": "no_workers", "retry_after": 1, "reason": "worker_crash_recovery"}
```

Client retries after 1 second. By then, warm restart is complete.

State/Event Payloads

Supervisor → Gateway (worker death notification via IPC):

```
{
  "type": "worker_status",
  "worker_id": "worker-0",
  "state": "dead",
  "exit_code": "0xC0000005",
  "timestamp": "2025-01-15T10:23:45.123Z"
}
```

Supervisor → Gateway (worker recovery notification):

```
{
  "type": "worker_status",
  "worker_id": "worker-0",
  "state": "idle",
  "model_loaded": "indic-tts-v3",
  "warm_restart": true,
  "restart_duration_ms": 480
}
```

Design Tradeoff Explanations

Decision	Chosen	Alternative	Rationale
No request replay from disk	Requests not	WAL-style request log	Request payloads are small (<1KB). Client already has the data. Persisting

	persisted		adds disk I/O to the hot path for a rare event.
Warm restart via page cache	Rely on OS page cache	Custom memory-mapped shared region	OS page cache is free — no code needed. Shared memory regions add complexity and IPC surface.
Worker_state.json on disk	Persist last-loaded model	Pass model hint via IPC only	Survives supervisor restart too. File is tiny (<1KB), written infrequently.
Fixed restart budget	5 restarts / 60s	Adaptive budget based on crash pattern	Simplicity. Fixed budget is predictable. Adaptive adds state machine complexity for marginal benefit.
No partial result recovery	Lost request, client retries	Checkpoint inference mid-request	Inference is stateless and fast (~50ms). Checkpointing adds latency to every request for a rare failure case.

Performance & Reliability

Metric	Value	Notes
Crash detection latency	~1-5ms	Kernel-mediated (waitpid / WaitForSingleObject)
Gateway re-routing	~5-15ms	Immediate on EPIPE detection
Worker respawn	~15-50ms	CreateProcess / posix_spawn
Warm model reload	~100-400ms	OS page cache hit via mmap
Total recovery time	~200-500ms	vs 3-5s cold start
Requests lost per crash	1 (the in-flight request)	Others rerouted or queued
Max crash rate before abandon	5 / 60s per worker	After: health alert, manual intervention

Reliability guarantees

- **No cascading failure:** worker crash doesn't affect gateway or other workers
- **At-most-once delivery:** the crashed request is lost, not duplicated
- **Bounded recovery time:** warm restart is 6-10× faster than cold start, capped at ~500ms
- **No data corruption:** inference is stateless, no mutable state shared between workers

9. Platform-Specific Detection and Cascading Crash Handling

9.1 OS-specific crash detection primitives

Platform	Primitive	Expected detection window	Operational note
Windows 11	WaitForSingleObject(process_handle, INFINITE) + GetExitCodeProcess	~1-5ms	Treat 0xC0000005 and 0xC0000409 as non-recoverable worker exits for the current process instance
macOS 14+	waitpid(pid, &status, 0)	~1-5ms	Distinguish SIGKILL (external terminate) from SIGSEGV (runtime fault) for diagnostics and restart reason

9.2 Cascading crash policy

If a replacement worker crashes during warm recovery:

1. Increment the same worker-slot restart budget counter.
2. Trigger exponential spawn delay for that slot (50ms , 100ms , 250ms , 500ms , cap 1000ms).
3. Keep sibling workers serving traffic with reduced capacity.
4. If budget is exceeded (5 crashes/60s), mark slot as `quarantined` and return deterministic `503 no_workers_for_slot` for affinity-routed requests.

This prevents oscillation where repeated immediate respawns amplify outage time.

10. Request Deduplication and Retry Contract

To preserve at-most-once semantics during crash recovery, requests are deduplicated by `request_id` in the gateway.

10.1 Deduplication rules

1. First arrival for `request_id` creates an in-flight record.
2. Replayed client request with same `request_id` while in-flight receives latest known state (`queued` or `running`) instead of re-execution.
3. Replayed client request with same `request_id` after completion returns cached terminal metadata for a short TTL (default 30s) and does not re-run inference.
4. Replayed client request with same `request_id` after terminal error returns same error payload until TTL expiry.

10.2 State payload example

```
{
  "request_id": "req_7f3a9b",
  "state": "running",
  "worker_slot": "worker-1",
  "retry_count": 1,
```

```
"deduplicated": true  
}
```

This ensures retries caused by client/network uncertainty do not multiply compute load during recovery.

Sarvam AI — Edge Runtime Team — Backend Intern Assignment